PROYECTO Nro1

Sistema Distribuido de Monitoreo de Sensores IoT

Internet: Arquitectura y Protocolos

Objetivo

Aplicar conceptos sobre servicios de aplicación y capa de transporte, en un proyecto que representa un sistema real, desplegado en una plataforma en la nube, haciendo uso de lenguajes de programación.

Introducción

En este proyecto se pondrán a prueba sus competencias en el diseño e implementación de aplicaciones distribuidas sobre Internet. A diferencia de algunas prácticas realizadas durante el curso, donde se implementaba un protocolo específico o se experimentaba con un servicio aislado, en esta actividad el objetivo es integrar múltiples servicios de red dentro de un mismo sistema funcional.

Como se ha revisado durante el curso, los servicios modernos de Internet rara vez dependen de un único protocolo. En la práctica, una aplicación distribuida utiliza varios servicios de forma coordinada, por ejemplo:

- resolución de nombres para localizar servicios
- autenticación de usuarios
- intercambio de datos en tiempo real
- servicios web para interacción con los usuarios
- infraestructura en la nube para el despliegue de servicios
- contenedores para facilitar el despliegue y la portabilidad de aplicaciones

En este curso, un protocolo puede entenderse como el conjunto de reglas que definen cómo dos programas se comunican entre sí. Estas reglas incluyen:

- el formato de los mensajes
- el orden en el que deben enviarse
- las respuestas esperadas
- el manejo de errores

El objetivo de este proyecto es que usted diseñe e implemente un sistema completo de monitoreo distribuido, donde múltiples clientes intercambien información con un servidor central utilizando sockets y un protocolo de aplicación diseñado por usted mismo.

Además, el sistema deberá ser desplegado en infraestructura de computación en la nube, utilizando servicios de Amazon Web Services (AWS) y empleando tecnologías de contenedores Docker, con el fin de simular el funcionamiento de una aplicación distribuida en un entorno real de Internet.

Tenga en cuenta que este es un proyecto que demanda una cantidad considerable de tiempo de diseño, programación, despliegue y pruebas, por lo que se recomienda iniciar su desarrollo lo antes posible.

Conceptos

Sockets y Abstracción

Los desarrolladores de aplicaciones utilizan abstracciones para simplificar el acceso a sistemas complejos. Una abstracción permite ocultar detalles internos del funcionamiento de un sistema y proporcionar una interfaz más simple para el programador.

Un socket es una de estas abstracciones. Puede entenderse como un punto de conexión que permite que dos aplicaciones intercambien información a través de la red. En términos prácticos, un programa puede:

- escribir datos en su socket
- mientras otro programa, en otra máquina o proceso, los recibe a través de su propio socket

Los sockets permiten que las aplicaciones se integren a la arquitectura de red TCP/IP sin necesidad de interactuar directamente con los protocolos de bajo nivel.

Tipos de Sockets en el Proyecto

Existen diferentes tipos de sockets, pero en este proyecto se utilizarán dos tipos de la API de Sockets Berkeley.

Sockets de flujo (SOCK_STREAM)

Proporcionan una conexión:

- fiable
- orientada a conexión
- con entrega ordenada de datos

Este tipo de socket suele utilizar el protocolo TCP, por lo que es adecuado cuando es necesario garantizar que los mensajes lleguen correctamente.

Sockets de datagrama (SOCK_DGRAM)

Proporcionan un servicio:

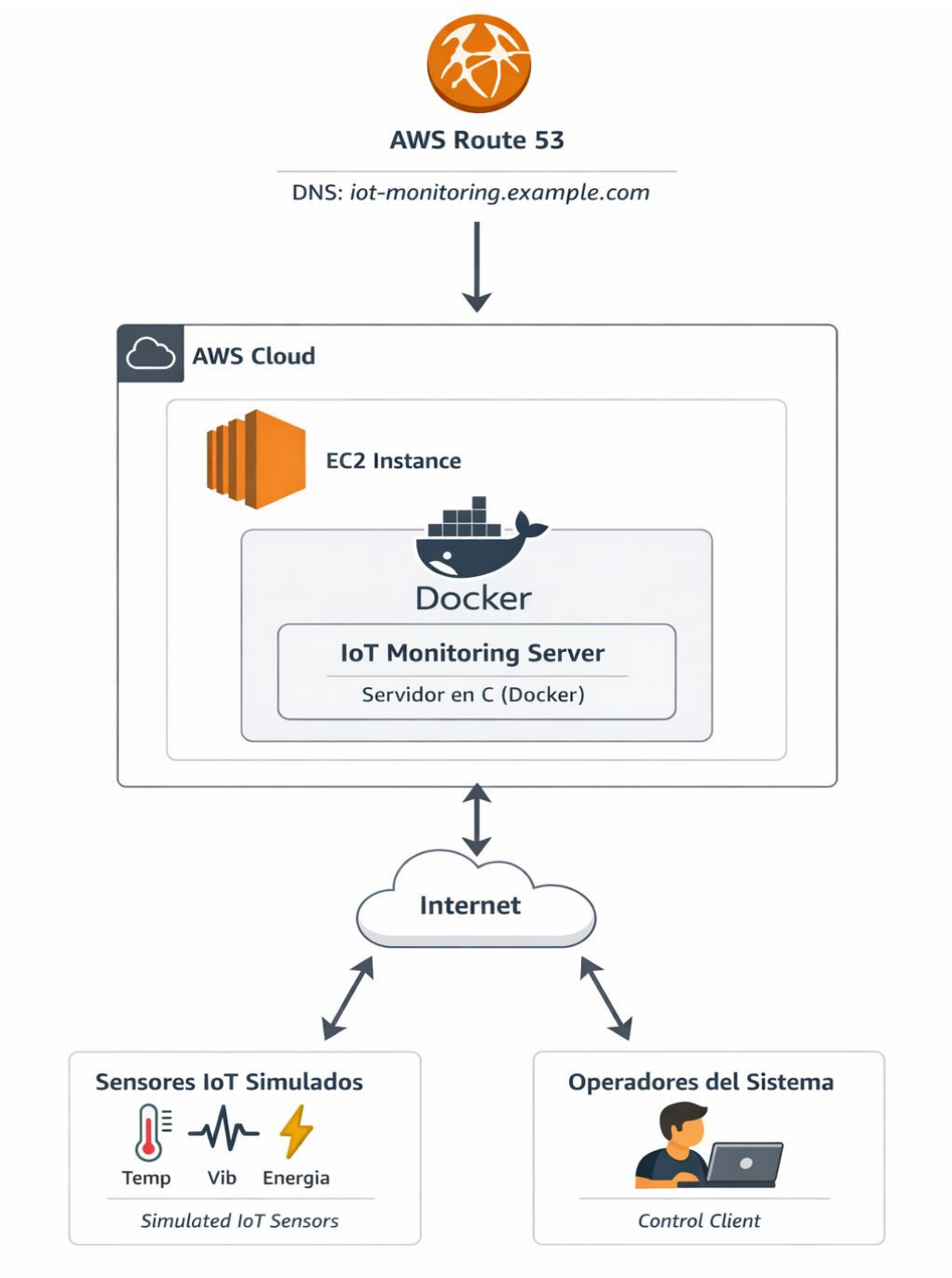
- sin conexión
- más rápido
- sin garantía de entrega

Este tipo de socket generalmente utiliza UDP y suele emplearse cuando la latencia es más importante que la confiabilidad.

Para este proyecto, usted deberá analizar las características del sistema que está construyendo y justificar qué tipo de socket utilizará en cada componente de la arquitectura.

Contexto

Una empresa dedicada al desarrollo de soluciones de infraestructura inteligente y automatización industrial desea crear una plataforma educativa para entrenar ingenieros en el monitoreo de sistemas IoT distribuidos.



En sistemas reales de monitoreo industrial, es común que cientos o miles de sensores distribuidos en diferentes ubicaciones reporten continuamente información sobre el estado de una infraestructura. Estos sensores pueden medir variables como:

- temperatura
- consumo energético
- vibración mecánica
- humedad
- estado operativo de equipos

Toda esta información es enviada a servidores centrales de monitoreo, donde es procesada para detectar situaciones anómalas o fallas potenciales. Actualmente, muchos de estos sistemas se ejecutan sobre infraestructura en la nube, lo que permite:

- escalar el número de dispositivos conectados
- garantizar alta disponibilidad del servicio
- acceder al sistema desde cualquier ubicación a través de Internet

Adicionalmente, el despliegue de aplicaciones modernas suele realizarse mediante contenedores, lo que permite empaquetar el software junto con sus dependencias y facilitar su ejecución en diferentes entornos.

Con el objetivo de entrenar estudiantes en el funcionamiento de este tipo de sistemas, la empresa desea construir un simulador interactivo de monitoreo IoT desplegado en la nube, donde diferentes participantes puedan asumir distintos roles dentro de la infraestructura.

El sistema estará compuesto por tres tipos principales de entidades.

Sensores

Los sensores representan dispositivos IoT que envían información periódicamente al servidor de monitoreo. Cada sensor pertenece a un tipo específico, por ejemplo:

- sensor de temperatura
- sensor de vibración
- sensor de consumo energético

Los sensores deben conectarse al servidor y enviar mediciones periódicas simuladas.

Operadores del sistema

Los operadores representan a los ingenieros encargados de supervisar el sistema. A través de una aplicación cliente, los operadores pueden:

- visualizar sensores activos
- recibir alertas generadas por el sistema
- consultar mediciones recientes
- ejecutar acciones de supervisión

Los operadores deben recibir notificaciones en tiempo real cuando ocurra un evento anómalo en el sistema.

Servidor central de monitoreo

El servidor central es responsable de:

- recibir datos de los sensores
- procesar las mediciones
- detectar eventos anómalos
- notificar a los operadores conectados

El servidor debe ser capaz de manejar múltiples sensores y operadores conectados simultáneamente.

Este servidor deberá desplegarse en una instancia de cómputo en AWS y ejecutarse dentro de un contenedor Docker, permitiendo que el sistema pueda ser desplegado de forma reproducible en diferentes entornos.

El sistema debe comportarse como un servicio real de Internet, por lo que debe cumplir con las siguientes características:

- los usuarios no se almacenan localmente en el servidor
- los servicios deben localizarse mediante resolución de nombres
- múltiples clientes pueden interactuar simultáneamente
- el sistema debe manejar errores de red sin finalizar su ejecución
- el servicio debe ser accesible desde Internet mediante infraestructura en la nube

Requerimientos

Interfaz Web

Los usuarios deben poder ingresar al sistema utilizando una interfaz web sencilla.

Dentro de esta interfaz podrán:

- iniciar sesión
- consultar el estado general del sistema
- visualizar los sensores activos

El servicio web debe implementarse mediante un servidor HTTP básico.

Este servidor debe ser capaz de:

- interpretar correctamente las cabeceras HTTP
- manejar peticiones GET
- devolver los códigos de estado correspondientes

Este servicio debe estar disponible a través de la instancia desplegada en AWS.

Servicio de autenticación

El sistema no debe almacenar usuarios localmente en el servidor principal.

Cuando un usuario intente iniciar sesión, el servidor debe consultar un servicio externo de identidad que determine:

- si el usuario existe
- qué rol tiene dentro del sistema

Resolución de nombres

El código del sistema no puede tener direcciones IP codificadas.

Todos los servicios deben ser localizados mediante resolución de nombres de dominio.

Para el despliegue del sistema en la nube, se deberá configurar un registro DNS que permita acceder al servidor mediante un nombre de dominio en lugar de una dirección IP.

Durante el desarrollo del proyecto se recomienda utilizar el servicio Amazon Route 53 para crear y gestionar este registro DNS.

Si la resolución falla, el sistema debe:

- manejar la excepción
- continuar su ejecución sin finalizar abruptamente

Protocolo de aplicación

Usted deberá diseñar, especificar e implementar un protocolo de la capa de aplicación que permita la comunicación entre:

- sensores
- operadores
- servidor de monitoreo

El protocolo debe ser basado en texto.

Este protocolo debe permitir operaciones como:

- registro de sensores
- envío de mediciones
- notificación de alertas
- consulta de estado del sistema

Despliegue en la nube

El sistema completo debe ser desplegado en AWS.

El despliegue debe incluir al menos:

- una instancia de cómputo (por ejemplo, EC2)
- configuración de puertos de red
- acceso remoto al servidor

El servidor debe ejecutarse dentro de un contenedor Docker, el cual debe incluir todas las dependencias necesarias para la ejecución de la aplicación.

El contenedor debe construirse utilizando un Dockerfile incluido en el repositorio del proyecto.

Durante la sustentación, los estudiantes deberán demostrar:

- la construcción de la imagen Docker
- la ejecución del contenedor en la instancia de AWS
- el acceso al sistema desde clientes externos

Detalles de la implementación

Clientes

Las aplicaciones cliente deben desarrollarse en al menos dos lenguajes de programación diferentes, por ejemplo: Python, Java, C++, Go.

Interfaz gráfica

El cliente operador debe tener una interfaz gráfica sencilla que muestre:

- sensores activos
- mediciones en tiempo real
- alertas generadas

Simulación de sensores

El sistema debe generar al menos cinco sensores simulados. Cada sensor debe:

- conectarse al servidor
- enviar mediciones periódicas
- identificarse con un tipo específico de sensor

Servidor

El servidor debe cumplir con las siguientes condiciones:

- debe estar implementado únicamente en C
- debe utilizar la API de Sockets Berkeley
- debe soportar múltiples clientes simultáneos
- se permite el uso de hilos para manejar concurrencia

El servidor debe ejecutarse dentro de un contenedor Docker desplegado en una instancia de AWS.

Logging

El servidor debe implementar un sistema de registro de eventos. Este debe registrar:

- peticiones entrantes

- respuestas enviadas
- errores ocurridos

Los registros deben incluir:

- dirección IP del cliente
- puerto de origen
- mensaje recibido
- respuesta enviada

Estos registros deben mostrarse en consola y almacenarse en un archivo de logs.

Ejecución del servidor

El servidor debe ejecutarse desde consola utilizando los parámetros:

./server puerto archivoDeLogs

Este proceso debe ejecutarse dentro del contenedor Docker desplegado en AWS.

Evaluación

La evaluación del proyecto se realizará mediante sustentación presencial del sistema desarrollado.

Los estudiantes deberán demostrar:

- el funcionamiento del sistema
- la arquitectura implementada
- el despliegue del servidor en AWS
- la construcción y ejecución del contenedor Docker

1. Programación del Servidor (30%)

- implementación de sockets
- manejo de múltiples clientes
- lógica del protocolo

2. Programación de Clientes (25%)

- conexión al servidor
- construcción correcta de mensajes
- procesamiento de respuestas

3. Funcionamiento del Sistema (20%)

- estabilidad del sistema
- coordinación de mensajes entre entidades

4. Despliegue en AWS y Docker (15%)

- configuración de la instancia
- construcción de la imagen Docker
- ejecución del contenedor
- acceso al sistema desde Internet

5. Documentación (10%)

- especificación completa del protocolo

Entrega

El proyecto debe desarrollarse en equipos. El código debe almacenarse en un repositorio privado.

La entrega debe incluir:

1. enlace al repositorio
2. código fuente del sistema
3. documentación del protocolo
4. archivo **Dockerfile**
5. instrucciones para el despliegue en AWS
6. configuración del dominio o registro DNS utilizado

Durante la sustentación se verificará que **no existan commits posteriores a la fecha límite de entrega**.